编译原理预习笔记：语法分析 (3.2.3 - 3.2.4)

- 3.1 上下文无关分析
  - 3.1.1 上下文无关文法的定义
    - 1. 为什么需要上下文无关文法?
      - 常用的语法分析方法：
        - 自上而下：根节点->叶节点（从左向右）
        - 自下而上：叶节点->根节点（从左向右）
        - 只能处理上下文法的子类
      - 正规式（Regular Expression）虽然能描述简单的语言，但无法描述具有**配对或嵌套结构**的语言。例如：
        - 配对括号组成的串：`((...))`
        - 嵌套结构：`if...then...else`
        - 某些重复串语言：$\{wcw \mid w \text{ 是 } a,b \text{ 的串}\}$
      - 上下文无关文法（Context-Free Grammar, CFG）比正规式表达能力更强，能够描述大多数编程语言的语法结构。
    - 2. 文法的四元组定义
      - 一个上下文无关文法 G 可以表示为一个四元组 $G = (V_T, V_N, S, P)$：
      - 1. V_T（终结符集合）：文法所定义的语言的基本符号。在编译器中，终结符通常被称为“记号名”（Token）。
      - 2. V_N（非终结符集合）：代表语法范畴的变量。非终结符集合与终结符集合不相交（$V_T \cap V_N = \emptyset$）。
      - 3. S（开始符号）：$S \in V_N$，是文法推导的起点，代表文法所定义的整个语言。
      - 4. P（产生式集合）：形式为 $A \to \alpha$ 的规则。其中 A 是非终结符（左部），α 是由终结符和非终结符组成的符号串（右部）。它说明了如何通过右部的序列来构造或替换左部的符号。
    - 3. 符号约定
      - **终结符**：小写字母（a, b, c）、运算符（$+, *$）、标点符号、数字、黑体字符串（**id**, **while**）。
      - **非终结符**：大写字母（A, B, C）、开始符号 S、小写斜体名字。
      - **文法符号**：大写字母 X, Y, Z（既可以是终结符也可以是非终结符）。
      - **缩写形式**：若 $A \to \alpha_1, A \to \alpha_2$ 等有相同左部，可简写为 $A \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_k$。
  - 3.1.2 推导 (Derivations)
    - **定义：** 推导是将产生式看作“重写规则”。如果文法中存在产生式 $A \to \gamma$，那么在任何包含 A 的符号串中，可以将 A 替换为 γ。
    - 1. 基本概念
      - **直接推导 ($\Rightarrow$)：** 一步替换。
        - e.g.: $\alpha A \beta \Rightarrow \alpha \gamma \beta$
      - 零步或多步推导($\xrightarrow{*}$)：经过 0 次或多次直接推导得到。
      - 一步或多步推导 ($\xrightarrow{+}$)：经过 1 次或多次直接推导得到。
    - 2. 句型与句子
      - **句型 (Sentential Form)**：如果 $S \xrightarrow{*} \alpha$，且 α包含终结符或非终结符，则称α为文法的句型。
      - **句子 (Sentence)**：如果句型中仅包含终结符，则称其为句子。
      - 语言 $L(G)$：文法 G 生成的所有句子的集合。
    - 3. 最左与最右推导
      - 在推导过程中，如果句型中有多个非终结符，需要决定替换顺序
      - **最左推导 ($\Rightarrow_{lm}$)：**在每一步推导中，总是选择句型中**最左边**的非终结符进行替换。
      - **最右推导 ($\Rightarrow_{rm}$)：**在每一步推导中，总是选择句型中**最右边**的非终结符进行替换（也称为规范推导）。
  - 3.1.3 分析树 (Parse Trees)
    - 分析树是推导过程的图形化表示。它能够直观地展示语法符号是如何按照产生式组合成更高级别的语法范畴的。
    - 1. 定义与结构
      - **根节点**：标号为文法的开始符号 S。
      - **内部节点**：必须是非终结符，代表应用了一个产生式。
      - **叶节点**：可以是终结符，也可以是非终结符（代表尚未推导完）。
      - **产生式的体现**：如果节点 A 有子节点 $X_1, X_2, \ldots, X_k$，则对应文法中的一个产生式 $A \to X_1 X_2 \ldots X_k$。
    - 2. 分析树与推导的关系
      - **抽象性**：分析树忽略了推导的顺序（即无论是先替换左边的非终结符还是右边的，最终生成的分析树是相同的）。
      - **边缘 (Yield)**：将分析树的叶节点从左到右排列，得到的符号串称为树的边缘。如果边缘仅由终结符组成，则它就是该文法生成的一个句子。
  - 3.1.4 二义性（Ambiguity）
    - 1、定义
      - 一个文法：若存在某个句子有**多棵不同的语法分析树**
      - 等价说法：存在多个**最左推导 / 最右推导**
    - 2、典型例子
      - id * id + id
      - 两种推导：
        - ⇒ 对应两棵不同语法树
        - (id * id) + id
        - id * (id + id)
    - 3、本质原因：运算符**优先级**和**结合性**未被文法明确规定
    - 4、关键结论
      - 二义性文法 ⇒ 无法唯一确定语义结构
      - 编译器难以处理
    - 5、注意点
      - 文法二义性 ≠ 语言二义性
      - 语言可能有无二义文法
      - 若所有文法都二义 ⇒ 语言才是二义的
    - 6、消除二义性的方法：
      - 明确优先级（如 * > +）
      - 引入新非终结符分层（E, T, F）
- 3.2 语言和文法
  - 3.2.1 正规式 vs 上下文无关文法（CFG）
    - 1、表达能力关系
      - 所有正规语言都能用CFG描述
      - 但CFG更强
      - 正规式 ⊂ 上下文无关文法
    - 2、示例
      - 可转换为CFG：A0 → aA0 | bA0 | aA1A1 → bA2A2 → ε
      - (a|b)*ab
    - 3、自动机对应关系
      - 正规式 ↔ 有限自动机（DFA/NFA）
      - CFG ↔ 下推自动机（PDA）
    - 4、关键思想
      - 每个状态 → 一个非终结符
      - 转移规则 → 产生式
      - 可以将 NFA机械转换为CFG
    - 5、转换规则总结
      - 状态 i →ⱼ（输入 a） ⇒ Ai → aAj
      - ε 转移 ⇒ Ai → Aj
      - 接受状态 ⇒ Ai → ε
  - 3.2.2 分离词法分析器的理由
    - 为什么要分离?
      - 📌 原因一：简化设计
        - 词法规则：简单（正则表达式）
        - 语法规则：复杂（CFG）
        - 分离后：各自专注，降低复杂度
      - 📌 原因二：提高效率
        - 词法分析：可用**有限自动机（DFA）高速处理**
        - 若混在一起：语法分析复杂度大幅上升
      - 📌 原因三：提高可读性
        - 正规式更适合描述：标识符、数字、关键字
        - CFG更适合描述：结构（表达式、语句）、分层更清晰
      - 📌 原因四：增强可移植性
        - 字符集 / 输入设备差异：
        - 语法层保持稳定
        - 可限制在词法分析阶段处理
      - 📌 原因五：利于模块化（编译器设计核心）
        - 前端划分：
          - 词法分析（Lexical Analysis）
          - 语法分析（Syntax Analysis）
        - 好处：
          - 易维护
          - 易优化
          - 易替换组件
  - 3.2.3 验证文法产生的语言
    - 如何从数学上证明一个文法 G 产生的语言 $L(G)$ 正好就是我们预期的那个语言。
    - 1. 证明方法：数学归纳法
      - **完备性**：预期语言中的每一个句子都能由该文法推导出来。
      - **健全性**：该文法推导出的每一个句子都属于预期语言。
    - 2. 实例分析：括号配对文法
      - 文法 S：$S \to (S)S \mid \epsilon$
      - **目标**：证明该文法产生的语言是所有“配对括号串”的集合。
      - **证明思路**：
        - **推导方向**：通过归纳推导步数（n 步推导），证明得到的所有终结符串都是配对的。
        - **构造方向**：根据括号串的长度（$2n$），将其拆分为 $(x)y$ 的形式，证明 x 和 y 均可由 S 推导得出。
    - 3. 学习要点
      - 在设计语言时，通过这种归纳思考方式，可以帮助开发者设计出逻辑严密、无遗漏的文法规则。
  - 3.2.4 适当的表达式文法
    - 本节讨论如何设计**无二义性**且能体现**运算优先级和结合性**的算术表达式文法。
    - **二义性**：同一个句子对应多棵不同的分析树。这会导致语义理解偏差（例如 $2 + 3 * 4$ 到底是 20 还是 14）。
    - 1. 消除二义性
      - **解决方法**：通过引入不同的非终结符来强制规定推导的层次。
    - 2. 设计原则：优先级与结合性
      - 构造模式：
        - **底层 (基本单元)** factor：因子，运算的最小单位（标识符或括号表达式） `factor -> id|(expr)`
        - **中层 (高优先级)** term：项，通常处理乘除法等高优先级运算 `term -> term * factor|factor`
        - **顶层 (低优先级)** expr：表达式，处理加减法等低优先级运算 `expr -> expr + term|term`
      - **优先级**：优先级越高的运算符，在文法层次中越深（离起始符号越远），在分析树中越靠近叶子节点。
      - **结合性**：
        - **左结合**：使用**左递归**规则（如 `A -> A alpha | beta`）。
        - **右结合**：使用**右递归**规则（如 `A -> alpha A | beta`）。
    - 3. 关键结论